

Invoixy — Plain English Technical Guide

Zero-Knowledge Technical Reference & Viva Guide | Invoixy GST SaaS Platform

Who is this for: Anyone who needs to explain the technical side of Invoixy in a college viva or project presentation, even with zero technical background.

Rule: No confusing jargon. Everything is explained using everyday real-world analogies.

1. The Big Picture: The Restaurant Analogy

If an examiner asks, "How does your whole app work from front to back?", use this simple restaurant story:

[Customer at Table] Fridge] (Web Browser)	----->	[The Waiter] (Server Action)	----->	[The Kitchen] (Next.js)	----->	[The Pantry / (Turso Database)
--	--------	-----------------------------------	--------	------------------------------	--------	------------------------------------

1. The Customer at the Table (Frontend / Browser): This is the screen you see—the buttons, textboxes, and live invoice preview. When a user clicks "Save Invoice", they are placing an order.
2. The Waiter (Server Action / API): Takes the invoice data from the screen and carries it safely to the kitchen.
3. The Kitchen / Chef (Backend / Next.js): Checks the data, does the GST math, makes sure the numbers add up, and prepares the invoice.
4. The Pantry / Filing Cabinet (Database / Turso): The permanent storage where all invoices, customers, and products are stored on shelves so they never get lost even if you close the laptop.

2. Every Technology Explained in Plain English

Here is every tool used in Invoixy and what to say if an examiner asks "Why did you use this?":

1. Next.js (Version 16) — "The Master House"

- What it is: In traditional web development, you had to build the frontend (screens) in one folder and the backend (server) in a completely separate folder. Next.js lets you build both frontend and backend together in one single project.
- Why we used it: It makes the app load very fast, handles page routing automatically, and runs on the cloud easily.

2. React (Version 19) — "The LEGO Blocks"

- What it is: React is a library for making interactive screens. Instead of writing one huge 2,000-line webpage, React lets us build small, reusable blocks like LEGO pieces (e.g., a "Button" block, an "Input Field" block, a "Card" block).
- Why we used it: Whenever you type a price or select a product, React instantly recalculates the preview on screen without having to refresh the whole webpage.

3. TypeScript — "The Spell-Checker / Bodyguard"

- What it is: Regular JavaScript is like writing an essay without spell-check—you only find your mistakes when the program crashes. TypeScript is a smart spell-checker that catches mistakes while you are typing code.
- Why we used it: Since we are dealing with money and taxes, a typo like typing a word instead of a number could break a customer's bill. TypeScript guarantees that numbers stay numbers.

4. Tailwind CSS (Version 4) — "The Paint and Styling Kit"

- What it is: CSS is the code that colors buttons, sets margins, and makes things look modern. Tailwind provides ready-made design utility classes (like `bg-white`, `rounded-xl`, `font-bold`).
- Why we used it: It allowed us to make Invoixy look like a clean, commercial software without writing messy custom CSS files.

5. Turso (LibSQL / SQLite) — "The Cloud Filing Cabinet"

- What it is: A database is just a digital filing cabinet. Traditional databases like MySQL require running heavy servers that cost money. Turso is a cloud-based, ultra-lightweight SQLite database hosted in AWS Mumbai.
- Why we used it: It is super fast (replies in less than 20 milliseconds), never goes down, and requires zero server maintenance.

6. Prisma ORM — "The Helpful Librarian / Translator"

- What it is: Usually, to talk to a database, programmers must write complex SQL commands (``SELECT * FROM Invoices WHERE...``). Prisma is like a helpful librarian who translates simple English-like code (``prisma.invoice.create(...)``) into database actions.
- Why we used it: It makes database saving safe, prevents hacking (SQL injection), and ensures our code is clean.

7. PDFKit — "The Virtual Document Printer"

- What it is: A tool running on our server that knows how to draw shapes, lines, text, and logos onto an official A4 page.
- Why we used it: If you just use the browser's "Print" button, every printer cuts off borders differently. PDFKit generates a permanent, identical, high-definition PDF file on the server that looks perfect on every phone, laptop, and printer.

8. Vercel — "The 24/7 Cloud Host"

- What it is: The global cloud platform where our code lives on the internet.
- Why we used it: It connects directly to our GitHub repository. Whenever we update code, Vercel automatically rebuilds and deploys the website live to ``https://invoixy.vercel.app`` in seconds.

3. What Actually Happens When You Click "Save Invoice"?

If an examiner asks, *"Walk me through the lifecycle of saving an invoice"*, say this simple 4-step story:

1. Step 1 (User clicks): The user fills in the customer and items and clicks the purple "Save Invoice" button on their browser.
2. Step 2 (The Server Action): The browser bundles the data into a package and sends it to our server using a Next.js Server Action.
3. Step 3 (The Math & Sequence Check): The server calculates the exact CGST, SGST, or IGST, checks the financial year sequence counter (**e.g., this is invoice #7**), and assigns ``APEX/26-27/7``.
4. Step 4 (Database Insertion): Prisma takes the package and writes it into the Turso cloud database inside an atomic transaction. The database says *"Saved!"*, and the screen shows a success toast notification.

4. The Database Explained (The 10 Drawers)

Our database has 10 tables. Think of them as 10 drawers in a filing cabinet:

1. ``User`` Drawer: Stores who logs in (Name, Google Email).
2. ``Shop`` Drawer: Stores the businesses (**e.g., Apex Electronics & Appliances**).
3. ``ShopMember`` Drawer: Remembers who owns which shop (**e.g., Umar is the OWNER of Apex Electronics**).
4. ``SellerSettings`` Drawer: Stores the shop's profile (GSTIN, PAN, Bank IFSC, and the store logo).
5. ``Customer`` Drawer: The saved phonebook of frequent buyers (**e.g., Rahul Sharma**).
6. ``Product`` Drawer: The price catalog of inventory (**e.g., Sony Bravia TV, MacBook Pro**).
7. ``Invoice`` Drawer: The permanent record of every completed bill (Grand total, dates, paid/unpaid status).
8. ``InvoiceLine`` Drawer: The individual items inside each bill (**e.g., Item 1: 1 TV at ₹64,990**).
9. ``InvoiceSequence`` Drawer: The counter that counts 1, 2, 3, 4 so no two invoices ever get the same number.
10. ``SecuritySettings`` Drawer: Remembers the auto-lock screen timer and security PIN.

5. Multi-Tenancy Explained: The Apartment Building Analogy

If an examiner asks, *"What is multi-tenancy? Can one user see another user's bills?"*:

Use this analogy:

"Think of Invoixy like an apartment building. All users live in the same building (the same database), but each user has their own private, locked apartment (shopId)."

When Umar logs in, the key only unlocks his apartment. He can only see his invoices. When another user logs in, they get their own apartment. The database code strictly checks the apartment number on every single request, making it impossible for users to see each other's bills."

6. Indian GST Rules in Plain English

If an examiner asks, *"How does your app calculate GST?"*:

- The 2-Letter State Code Rule:
- * In India, every state has a 2-digit GST code. Maharashtra is code 27.
- * Intra-State (Same State): If our store is in Maharashtra (27) and the buyer is in Maharashtra (27), the law says tax must be split 50% to the Central Government (CGST) and 50% to the State Government (SGST). (e.g., an 18% tax becomes 9% CGST + 9% SGST).
- * Inter-State (Different State): If the customer is from Gujarat (24), the tax cannot be split between state governments. It all goes to Integrated GST (IGST) at the full 18%.
- The Round-Off Rule:
- * Customers don't carry 23 paise coins. If a bill total is ₹194,399.78, our code automatically rounds it up to ₹194,400.00 and records the ₹0.22 as "Round off".
-

7. How Login Works: The VIP Wristband Analogy

If an examiner asks, *"How does your authentication work?"*:

"When a user logs in with Google or Email, our server generates a digital VIP wristband called a JWT (JSON Web Token). We seal this wristband inside a special secure cookie called an HttpOnly cookie. Every time the user clicks a page, the browser shows this wristband to the server. Because it is marked 'HttpOnly', hackers cannot steal it using malicious JavaScript in the browser."

8. Top 10 Viva Questions & The Exact Plain-English Answers

Memorize these simple, powerful answers for your viva:

Q1. What is the tech stack of your project?

Say: *"We built Invoixy using Next.js 16 and React 19 for both frontend and backend, TypeScript for error-free coding, Tailwind CSS for modern styling, Prisma ORM for database queries, and Turso cloud database for storing data."*

Q2. Is this frontend or backend?

Say: *"It is a Full-Stack application. Next.js handles both the interactive frontend screens and the server-side backend logic in one unified codebase."*

Q3. Where is your database hosted?

Say: *"Our database is hosted on Turso, which runs on AWS Mumbai servers. It is a serverless SQLite engine that gives us ultra-fast query responses in under 20 milliseconds."*

Q4. What is Prisma and why did you use it?

Say: *"Prisma is an Object-Relational Mapper (ORM). It acts like a translator between our TypeScript code and the database, so we don't have to write raw SQL queries, and it stops SQL injection hacking automatically."*

Q5. How does your app prevent duplicate invoice numbers?

Say: *"We have an `InvoiceSequence` table inside an atomic database transaction. Every time an invoice is created, it locks the counter, increments by 1, and assigns a unique number for that financial year."*

Q6. How do you generate the PDF?

Say: *"We use PDFKit on the server. Instead of relying on the browser's print dialog—which can look different on different printers—our server draws the exact A4 invoice, embeds the logo, and outputs a standard PDF file."*

Q7. How does the Demo mode work?

Say: *"The demo button mounts a shared central sandbox store called 'Apex Electronics (Demo)'. It is pre-loaded with invoices and products so anyone—including an examiner on their phone—can test the app instantly without having to create an account."*

Q8. What happens if the internet goes down?

| Say: *"Invoixy has a dual-database design. In local mode on a laptop, it runs completely offline using a local SQLite file (dev.db). In production, it connects to Turso cloud."*

Q9. What security features did you implement?

| Say: *"We implemented three security layers: first, cryptographically signed HttpOnly JWT cookies; second, multi-tenant database isolation so users only see their own store; and third, an inactivity auto-lock screen for retail counters."*

Q10. What is the biggest advantage of Invoixy over traditional billing?

| Say: *"Speed and accuracy. With 1-click presets and automated GST math, a shopkeeper can generate an official, error-free tax invoice in under 30 seconds instead of spending 5 minutes writing on paper or spreadsheets."*

